

CASE STUDY 3

Stabilize Sprint Execution & Improve Delivery Predictability

Managing Sprint Delivery for GDT

THE PROBLEM

It's not a process gap. It's a **translation gap**.
Agile is a mindset. The team is treating it as a process.

THE SYMPTOMS:



Stories enter
without clarity



Acceptance Criteria
changes
late in sprint



Design & eng
interpret differently



Ad hoc work
added mid-sprint



Carry-over
increasing



Team morale
dipping

The people writing stories, designing them, and building them are each solving a slightly different problem. The team is following agile rituals but missing the principle: **shared understanding before commitment**, continuous feedback during execution.

ASSUMPTIONS

- GDT runs 2-week sprints. Team: 3-5 engineers, 1 designer, 1 PM.
- Lightweight agile process (Jira/Linear + standups) but ceremonies are informal and inconsistent.
- Founder deeply involved in day-to-day decisions; frequently shifts priorities based on market signals.
- No formal backlog grooming or refinement. Stories handed to engineering at sprint planning.
- No QA role. Engineers self-test. Bugs found post-merge cause rework.
- Retrospectives sporadic and don't produce actionable changes.
- Team is motivated but frustrated. They want to ship, not argue about process.
- Data points for better scope changes and understanding

ROOT CAUSE DIAGNOSIS

Six Causes, One Compounding Chain

1 Translation Gap
Stories are intent, not specification. PM says "booking flow." Engineering builds a different picture. Gap surfaces mid-sprint as scope changes.

2 No Readiness Gate
Sprint planning pulls unrefined stories. Ambiguity isn't resolved until implementation, when it's expensive to change.

3 Open Door Problem
Founder/stakeholders add work mid-sprint without removing anything. 20-30% of capacity consumed by unplanned work.

4 No Testing Mindset
Engineers self-test, but builders check 'does it work?' Testers ask 'how does it break?' Bugs post-merge = rework.

5 No Feedback Loop
Retros are sporadic, unfocused, or produce generic actions. Same failure modes repeat sprint after sprint.

6 Invisible Progress
Blockers aren't surfaced until Day 8. Standups are status reports, not problem-solving. Late discovery = carry-over.

Each cause feeds the next. #4 (no testing mindset) is the gap most teams miss — builder ≠ breaker.

INITIAL SPRINT PLAN

Two Sprints to Predictability

SPRINT 0: STOP THE BLEEDING

This sprint. No new ceremonies. Three changes.

● The Two-Engineer Test

Ask two engineers separately: "Describe done for this story." If answers don't match, clarify before more code is written. Takes 30 min total.

Fixes: Root Cause #1

● Add Testing Practice

Have someone in the team who does the testing, and has a breaker mindset. They should test as to how can the system break.

Fixes: Root Cause #4

● Blockers-First Standup

Open with: "What's stopping you today?" Not "What did you do yesterday?" PM owns resolving blockers by EOD.

Fixes: Root Cause #6

SPRINT 1: BUILD THE RHYTHM

Next sprint. Four structural improvements.

● Wednesday Refinement

PM + designer + 1 engineer, 1hr/week. Refine top stories for NEXT sprint. Stories get "Ready" tag.

Fixes: Root Causes #1, #2

● Planning with Ready Bar

Only "Ready" stories enter sprint. Commit to 70-80% capacity. Deliberate buffer for unplanned work.

Fixes: Root Causes #2, #3

● One-Action Retro

One question: biggest slowdown. Vote. One action, one owner, one deadline. Review last action. 30 min.

Fixes: Root Cause #5

● Sprint Health Board

Backlog → Ready → In Progress → Review → Done. PM checks 2x daily. Story stuck 48hrs = proactive check-in.

Fixes: Root Cause #6

DAY-TO-DAY PROCESS

Five Practices, 2.5 Hours/Week

Wednesday Refinement

1hr/wk

PM + Designer + 1 Eng (rotating) | Wed

Walk top 5-7 stories for next sprint. Engineer asks: "What would I need to start Monday?" Unanswered questions = PM tasks. Stories tagged: Ready / Needs Design / Not Ready.

Blockers-First Standup

50min/wk

Full team | Daily, 10 min max

PM reads board: "These haven't moved." Each person: "Am I blocked?" PM owns resolution by EOD. No "what I did yesterday." The board shows that.

Sprint Planning + Buffer

30min/wk avg

Full team | Sprint Day 1

Only 'Ready' stories enter. T-shirt sizes (S/M/L). Fill to 70-80% capacity. 20-30% = velocity buffer for ad-hoc and bugs.

Test

Built In

Team Member | Before Merge

Only 'Ready' stories enter. T-shirt sizes (S/M/L). Fill to 70-80% capacity. 20-30% = velocity buffer for ad-hoc and bugs.

One-Action Retro

15min/wk avg

Full team | Sprint last day, biweekly

One question: "Biggest slowdown?" Vote. Discuss top issue only. One action, one owner, one deadline. Review last sprint's action first. Done in 30 min.

TOTAL: ~ 2.5 hrs/week PM | ~ 1.5 hrs/week engineers | Half of standard scrum overhead

MANAGING AD HOC REQUESTS

Plan Capacity Buffer

Plan at 70-80% capacity. The remaining 20-30% is a deliberate buffer — not slack, but planned flexibility.

1 Measure Velocity

Track actual completed work over 2-3 sprints.
Establish baseline capacity.



2 Plan to 70–80%

Commit planned stories to 70-80% of
measured velocity. Buffer is intentional.



3 Buffer Absorbs

Ad-hoc requests, bugs, unknowns flow
into the buffer — no trade needed.

THE THREE LANES:

Production Fire

Revenue loss, data loss,
security issue

Drop everything.
Fix now. Buffer or not.

*Payment gateway
returning 500s*

Buffer Fit

Fits in remaining buffer.
Can be sized quickly.

Absorb into buffer.
No trade needed.

*Quick config change
for partner launch*

Next Sprint

Important but buffer full.
Not time-critical.

Acknowledge, backlog.
Prioritize at planning.

*Commission
breakdown page*

RISKS, TRADE-OFFS & SUCCESS METRICS

SUCCESS METRICS

Sprint Completion Rate	~55% → 80%+	Primary. The single number for predictability.
Carry-Over Count	3-5 → 1-2 max	The symptom the team feels most.
Unplanned Work Ratio	~30% → <20%	Quantifies the open door problem.
Bug Escape Rate	Track → <10%	Bugs after merge. Tests breaker mindset.
Story Readiness	Target 90%+	% of sprint stories tagged "Ready" at planning.
Retro Action Done?	Yes, every sprint	Meta-metric: is the improvement loop working?

TRADE-OFFS

Choosing: Buffer (absorb change) Over: Scope Freez (resist change) <i>Agile-native</i>	Choosing: Testing Member Over: QA Automation or QA Engineer <i>Lean, builds empathy</i>	Choosing: One retro action per sprint Over: Comprehensive action list <i>Execution over ambition</i>	Choosing: 70-80% capacity planning Over: 100% utilization <i>Buffer absorbs interruptions</i>
--	---	--	---

KEY RISKS

Buffer seen as slack [High]

Track buffer utilization. Show data: buffer absorbed X requests this sprint.

Team resists cross-testing [Med]

Start small: one story/sprint. Show rework reduction. Build habit.

Refinement becomes bloated [Med]

Cap at 1hr, one engineer. PM preps beforehand. If it bloats, cut scope.

Completion rate doesn't improve [Med]

If flat after Sprint 2, root cause is sizing, not process. Switch to smaller stories.

This works for 5-10 people. At 15+: expand refinement, add sprint reviews, add QA process. Revisit in 6 months.